Public

ELISA
Enabling Linux in Safety Applications

ELISA Workshop London, UK

June 9-11, 2026
Co-hosted with Canonical

ELISA
Enabling Linux in Safety Applications

Functional Safety with Xen, Zephyr, and Linux

[Matthew Weber, Ayan Kumar Halder]

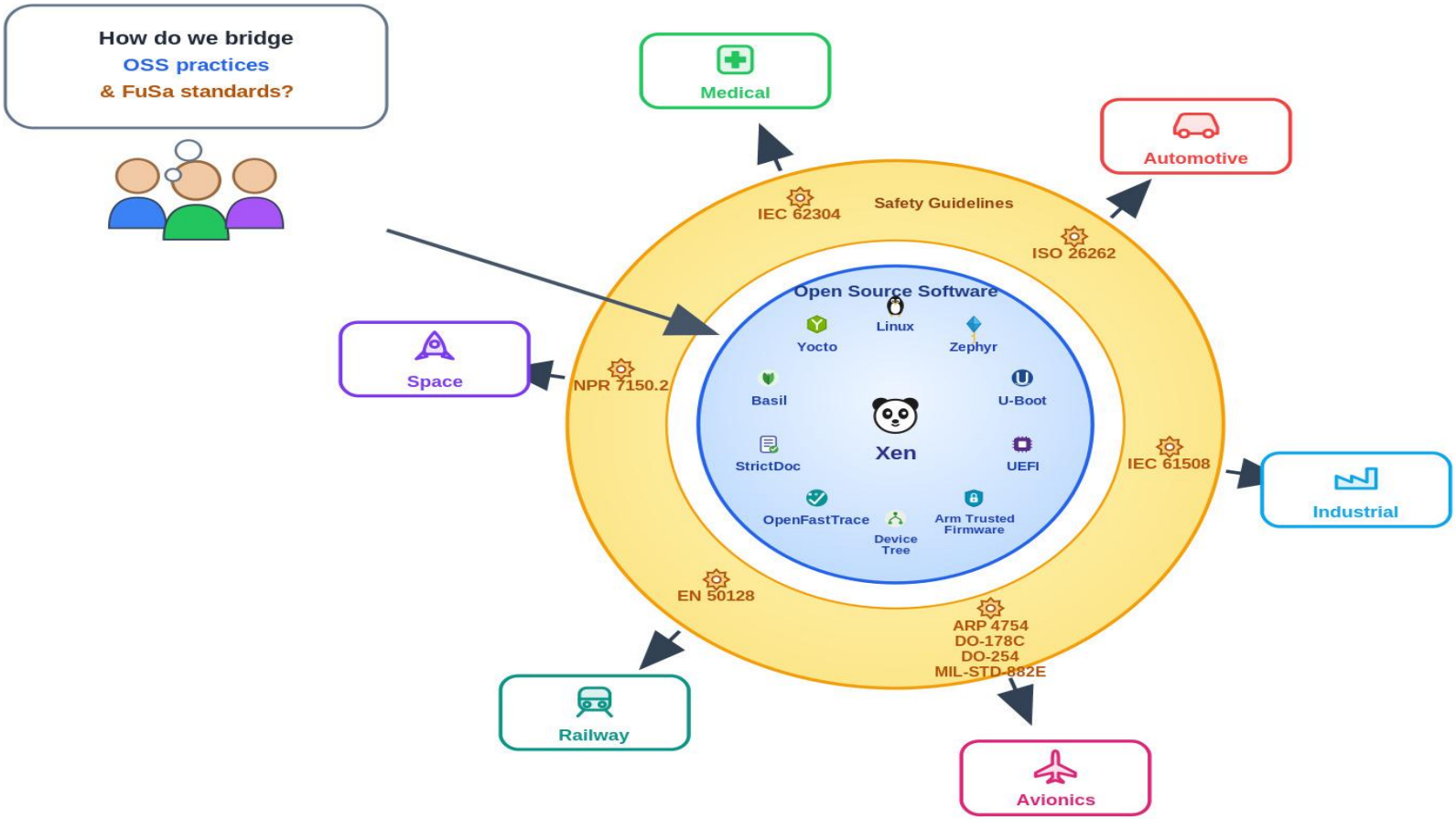
ELISA Workshop: London, UK

June 9-11, 2026
Co-hosted with Canonical

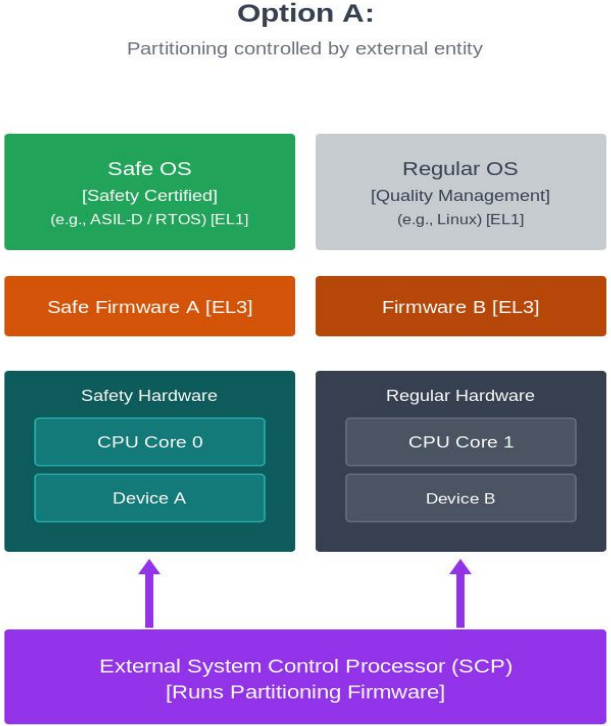
The flow

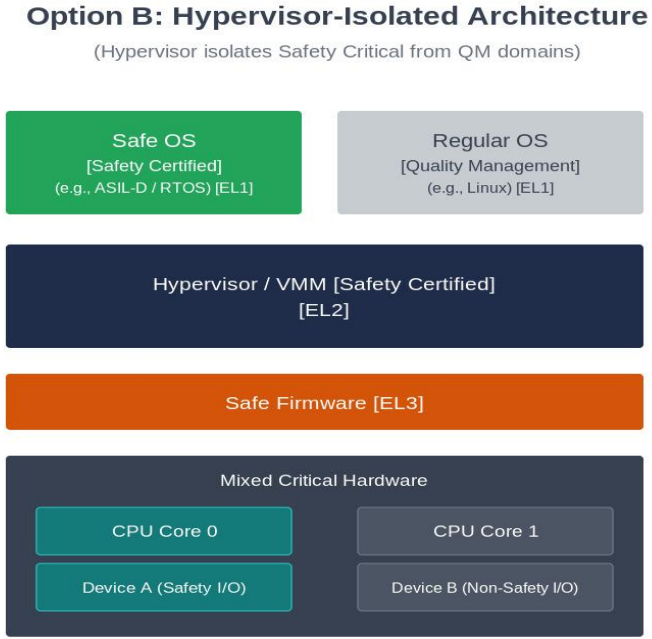
- Why use a hypervisor?
  - Key features of Xen which makes it useful for safety critical system
  - How Xen shares responsibility with Linux and Zephyr to enable a safety system
- It all begins with requirements
  - Describing the features that Xen provides to the domain - in our case market requirements
  - The flow from market to product to SSR
  - Why AoUs and the key AoUs
  - Why Arch spec
- How do you verify and validate a complex system?
  - The various kinds of tests - req based tests, BVA, fault injection, runtime
  - The tooling around - MCDC coverage
  - Further splitting the requirements to tie them to functions - making DO
- The Big question - evergreen certifiability - upstream, maintain
  - So Join us - Xen Project, Xen Safety Committee

Our shared vision - OSS meets FuSa



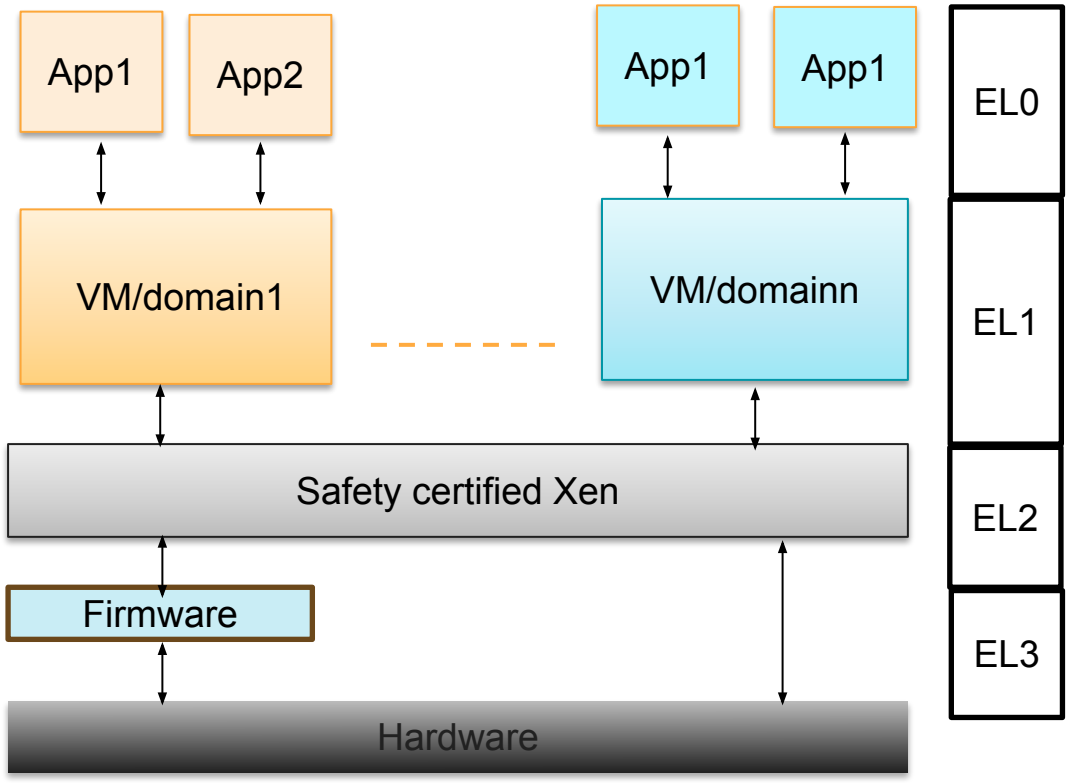
Safety certification approach - Need for hypervisor





Key Features of Xen for safety certification

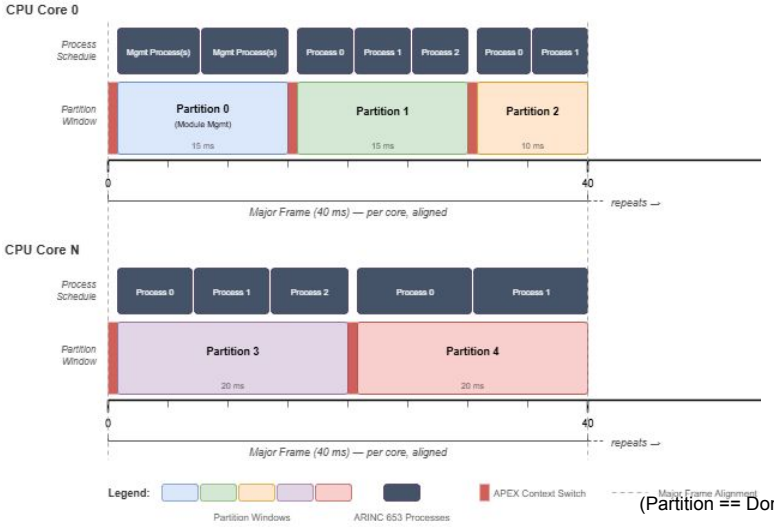
- Type1 hypervisor (Or Baremetal Hypervisor)
- Xen manages few hardware resources like Interrupt controller, IOMMU, Timer, UART, MMU
- **Partitioning** of CPU cores, memory and hardware resources to VMs. Ensures that the other VM will not have access to the device and memory
- Xen can create and boot domains in parallel (Hyperlaunch). There is no dependency on Dom0 for FuSa VMs
- **Supports Arm safety cores and IPs:** *Cortex-A720AE, GIC-600AE, MMU-600AE*



Different Schedulers supported - Time partitioning

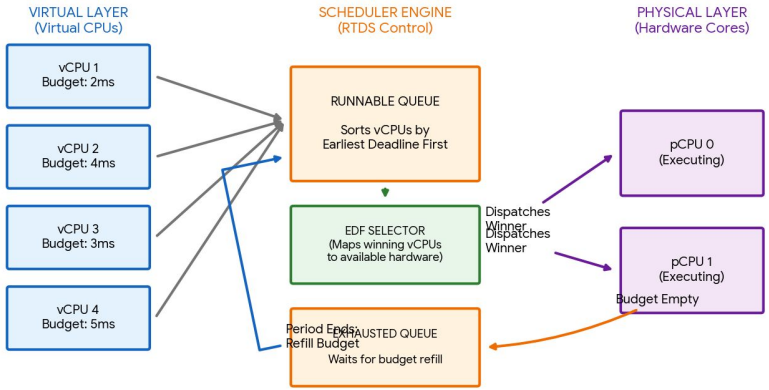
NULL – static pinning

ARINC653

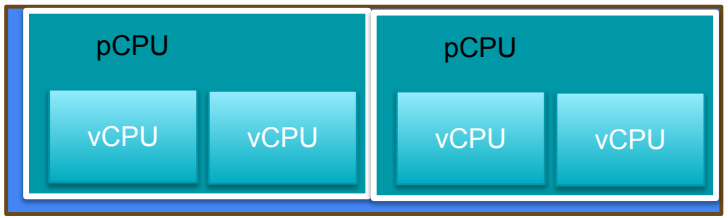


(Partition == Domain)

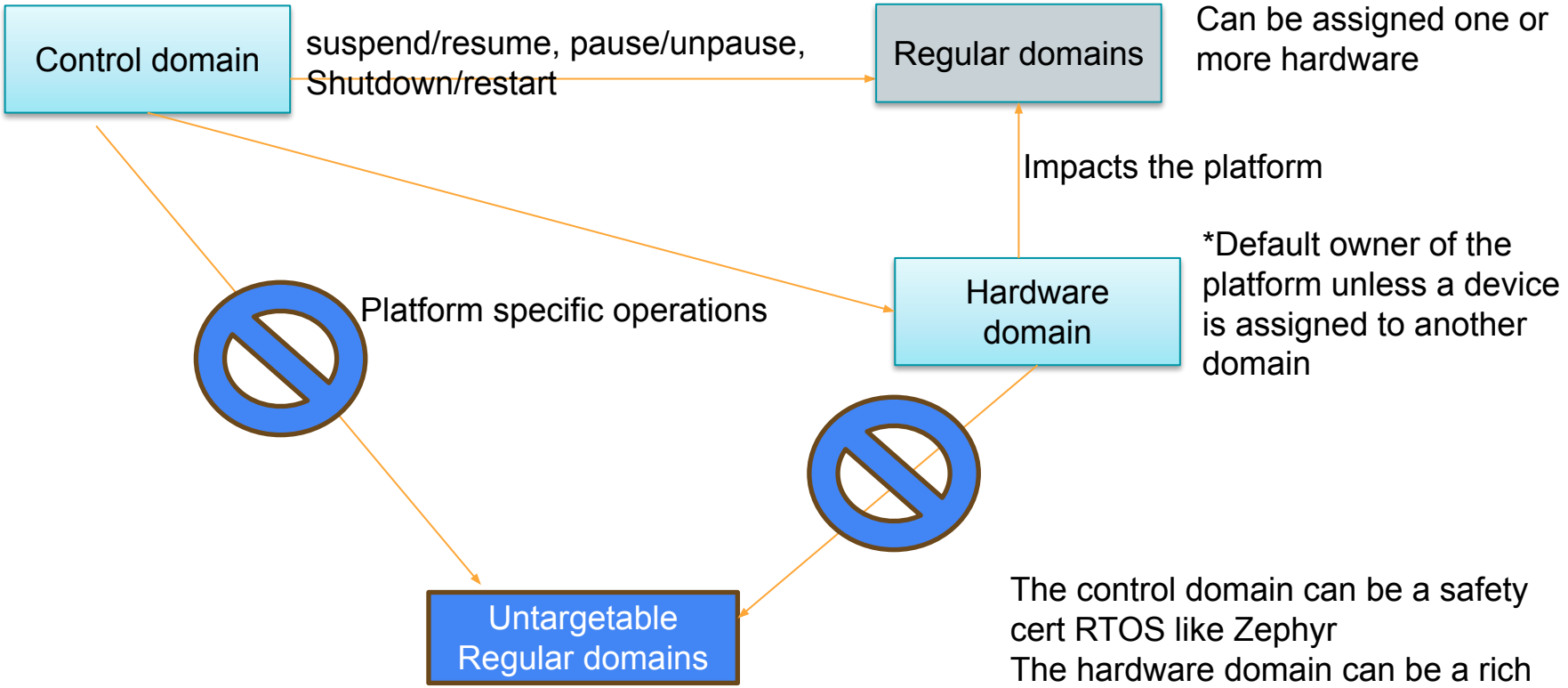
RTDS



Credit2

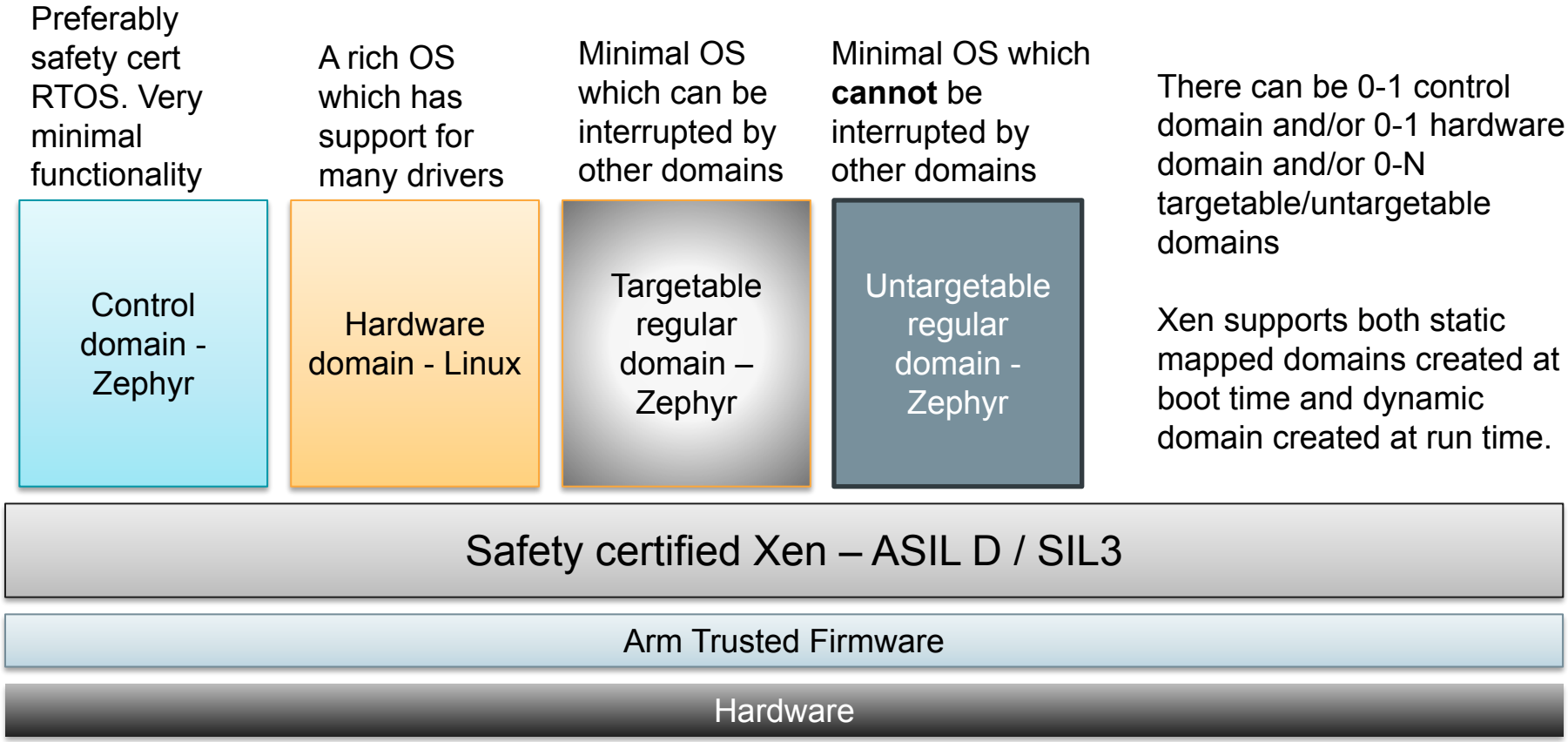


Xen relies on Domains to manage the system



The control domain can be a safety cert RTOS like Zephyr
The hardware domain can be a rich OS which supports many drivers like Linux

Xen with Zephyr and Linux as VMs



Assumptions on domains

Control domain

- Has the toolstack to create/destroy domains at runtime.
- Can run privileged operations to impact other domains (via mapping memory, etc)
- Should be untargetable

Hardware domain

- Can be a part of control domain (ie dom0) or separate.
- Able to support all the platform devices
- Can run device emulation (backend)
- Should be targetable

Regular domain

- Can be assigned one or more platform devices.
- Can share devices using PV or VirtIO with other domains.
- Can be targetable (for PV/VirtIO) or untargetable

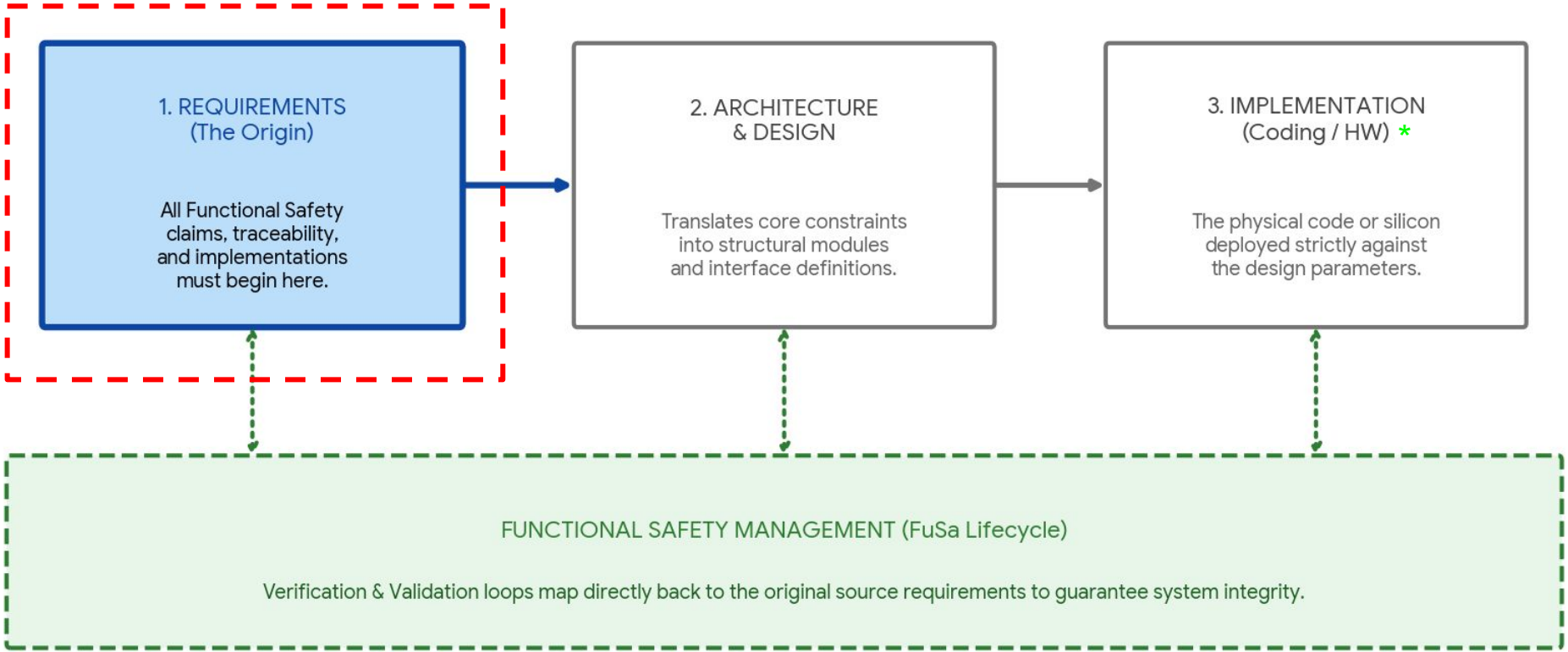
FUSA hardening with MISRA, DCE

- Directives 1.1, 2.1, 4.1, 4.7, 4.10, 4.11, 4.14 adopted
- 120 MISRA C rules adopted
  - https://gitlab.com/xen-project/xen/-/blob/staging/docs/misra/rules.rst?ref_type=heads
- ÉCLAIR MISRA C scanner used in upstream CI loop.
- Every patch submission goes through the MISRA C scans to avoid regressions.
- **C language extensions and the assumptions on the toolchain (GCC) are documented**

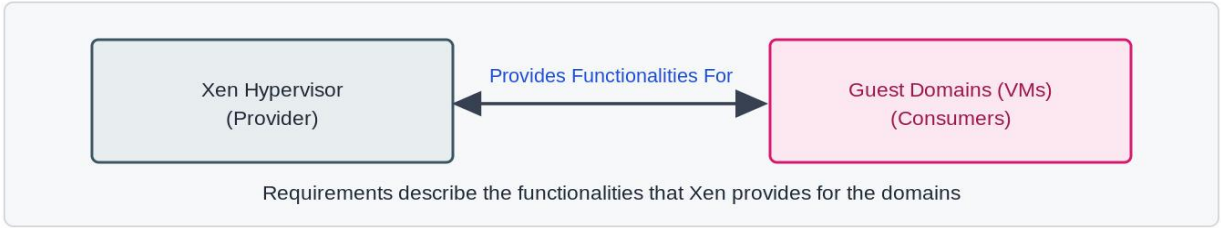
- Dead code elimination using granular KConfig, compiler nuances (constant conditionals) and VRP
- The aim is to ensure the minimal footprint of Xen for the specific platform while making the code configurable to support as wide range of platforms
- Enables upstreaming and common code between FuSa & non FuSa players
- Refer “[PATCH v2] xen: gic-v3: Introduce CONFIG_GICV3_NR_LRS ” on xen-devel

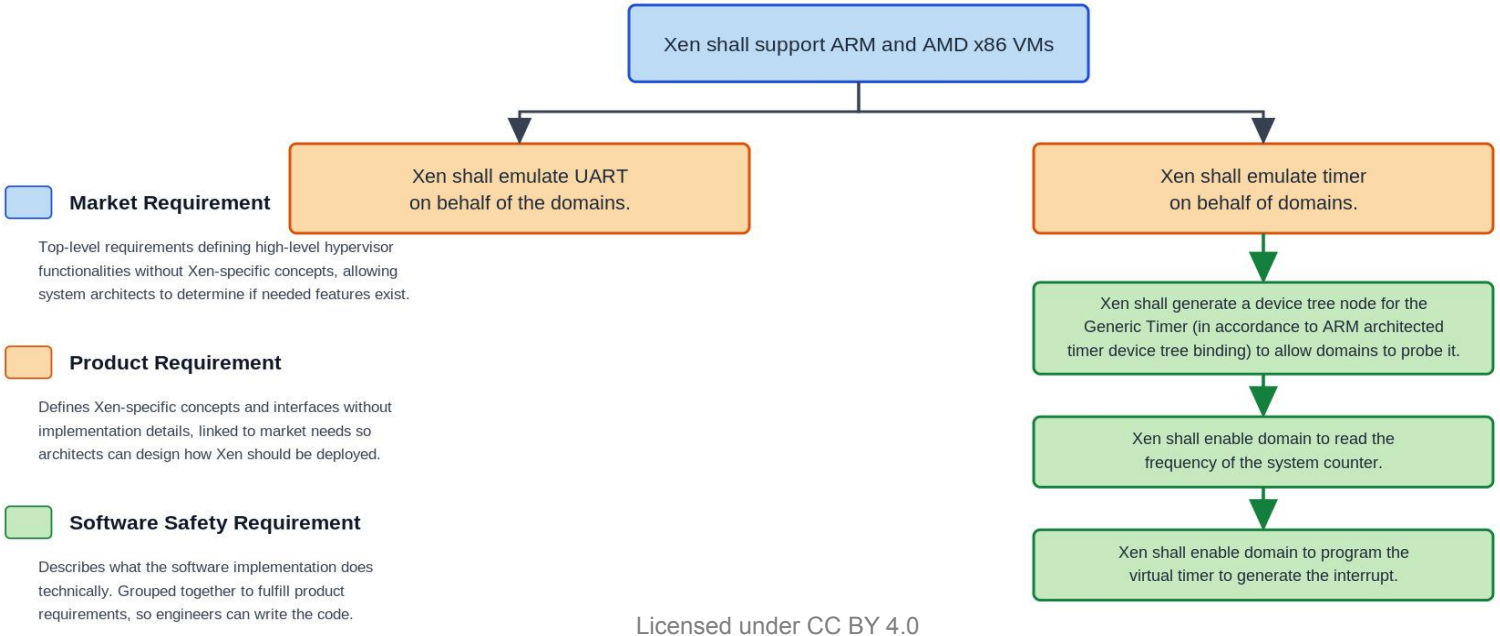
It all begins with requirements

*Code exists and drives requirements (IEC61508 Route 3S “Reuse”)

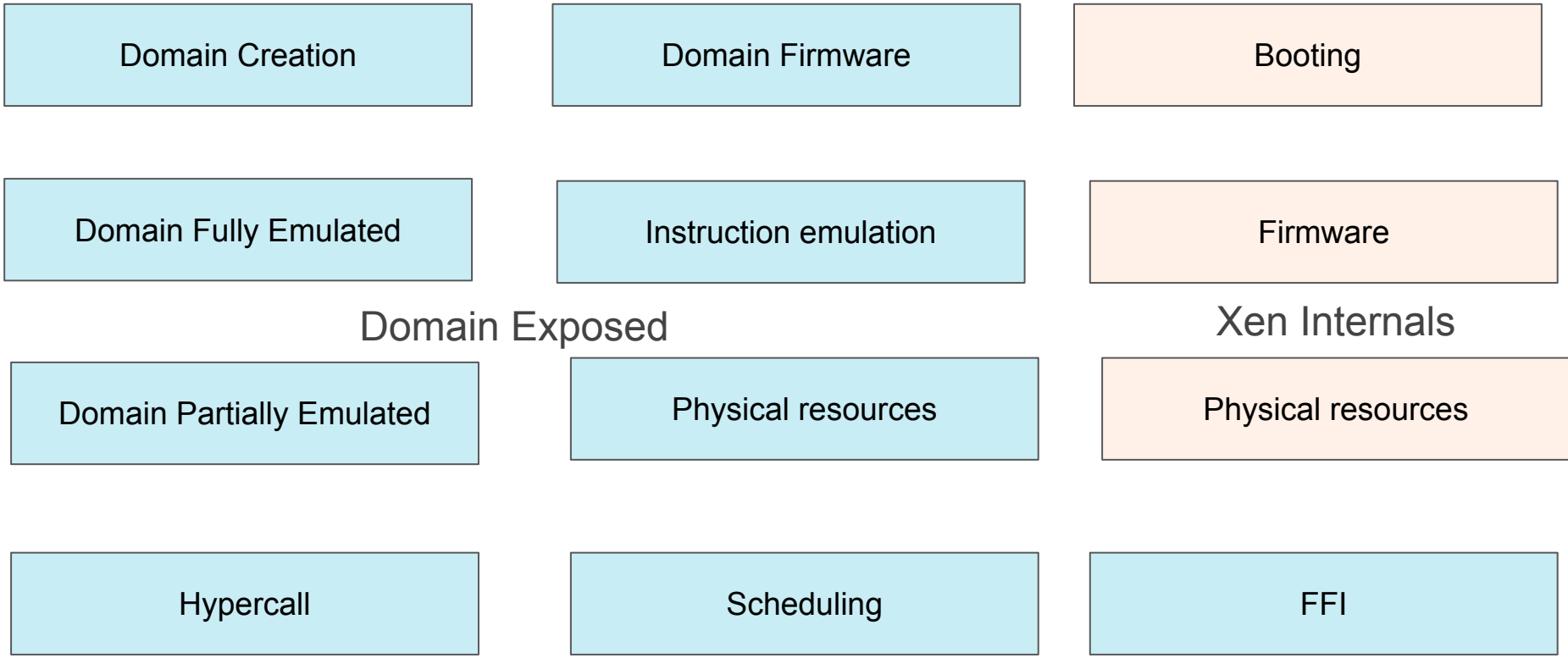


How do we define requirements in Xen





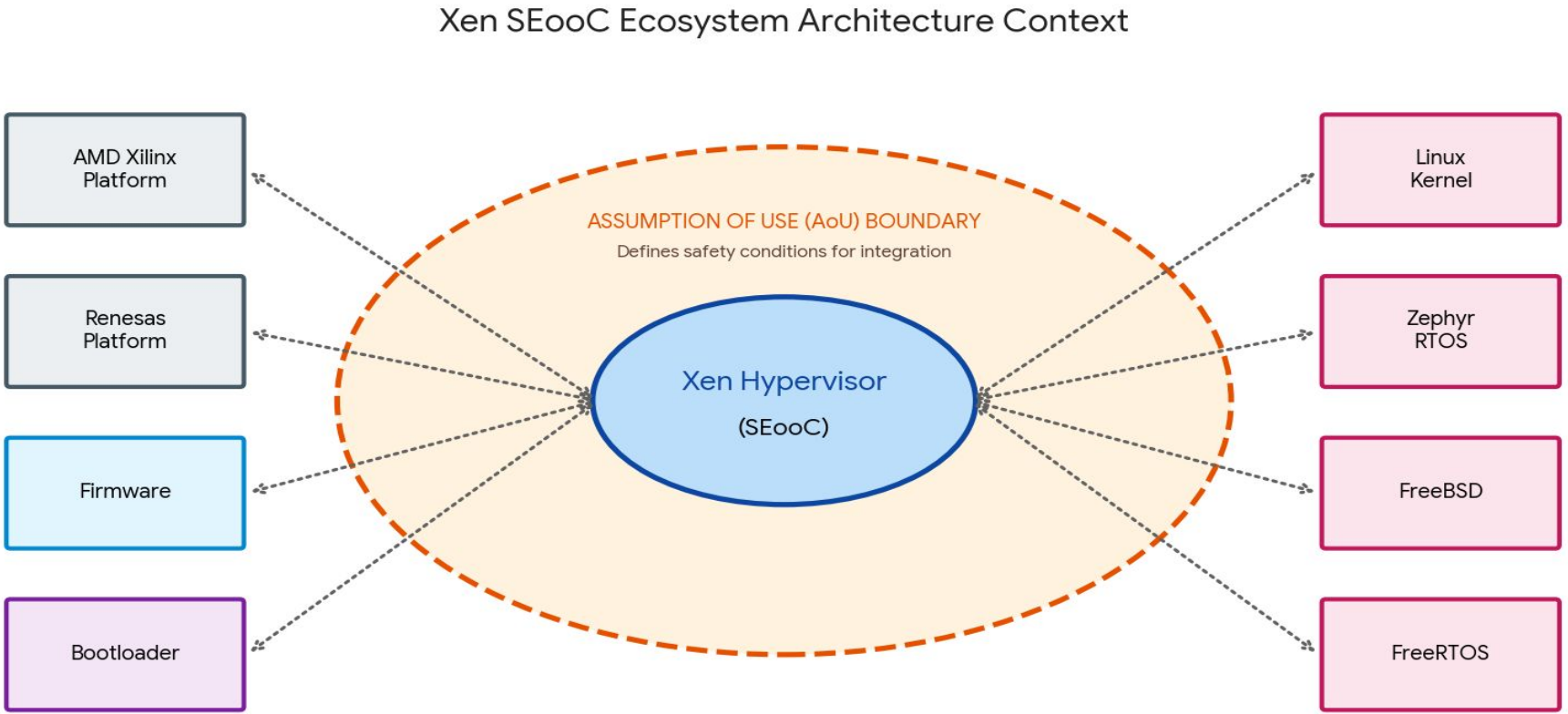
Organizing the software safety requirements



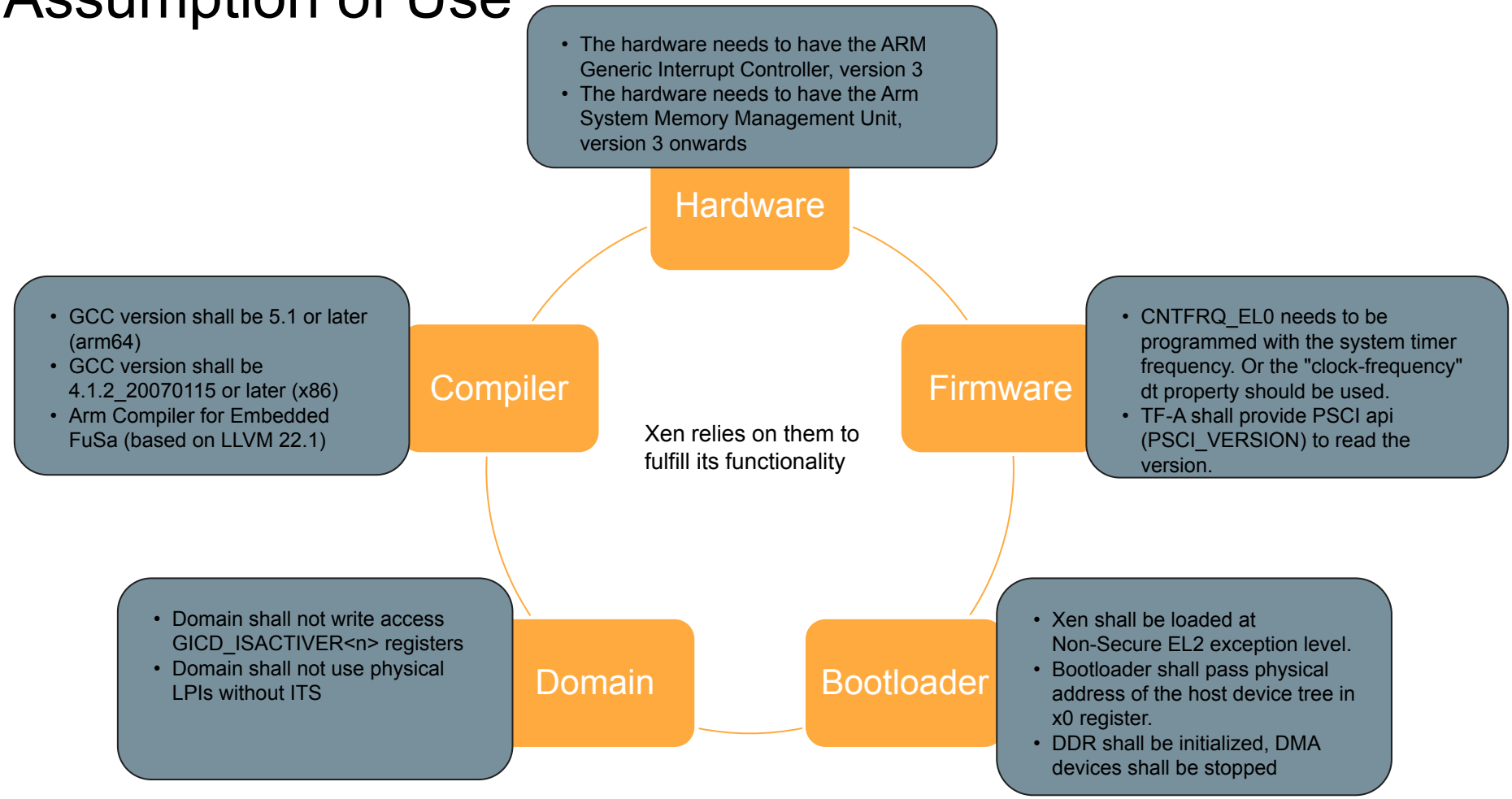
https://gitlab.com/xen-project/xen/-/tree/staging/docs/fusa?ref_type=heads
https://gitlab.com/xen-project/fusa/requirements

Remember Xen is SEooC

(Safety Element out of Context)



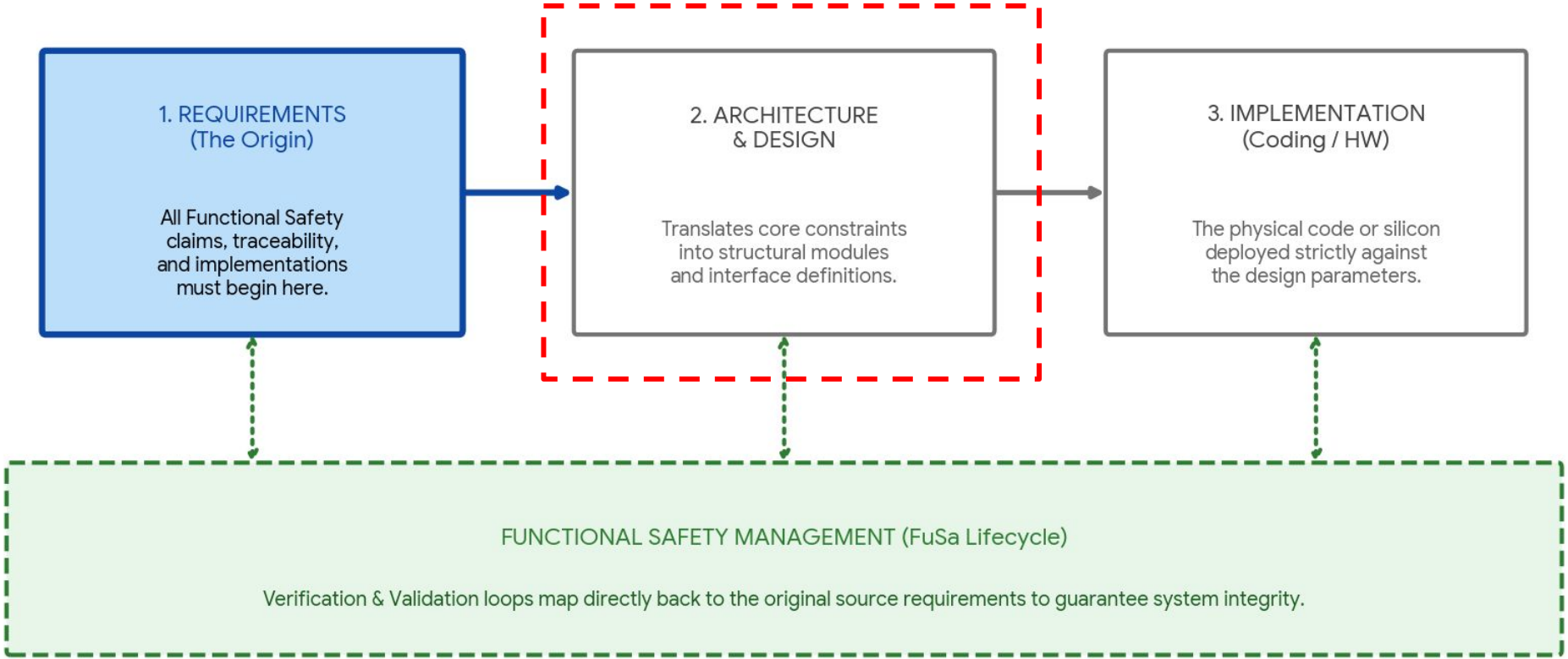
Assumption of Use



Example: Key Assumption of Use on Domains

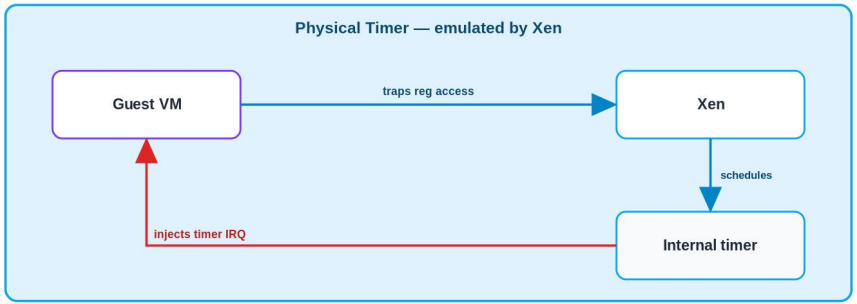
- On Zephyr as safety critical domain – It should mask its event channel notifications during the safety critical task to avoid being interrupted by other domains
- On Linux as QM domain – It shall enable the platform devices and perform platform operations (suspend, power off, reset, reboot) only when requested by the control domain.
    - It shall enable the PV.VirtIO backend drivers to perform device emulation

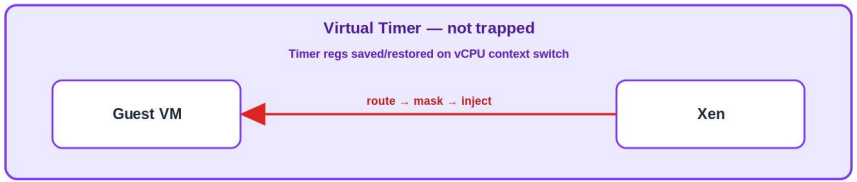
Architecture and Design



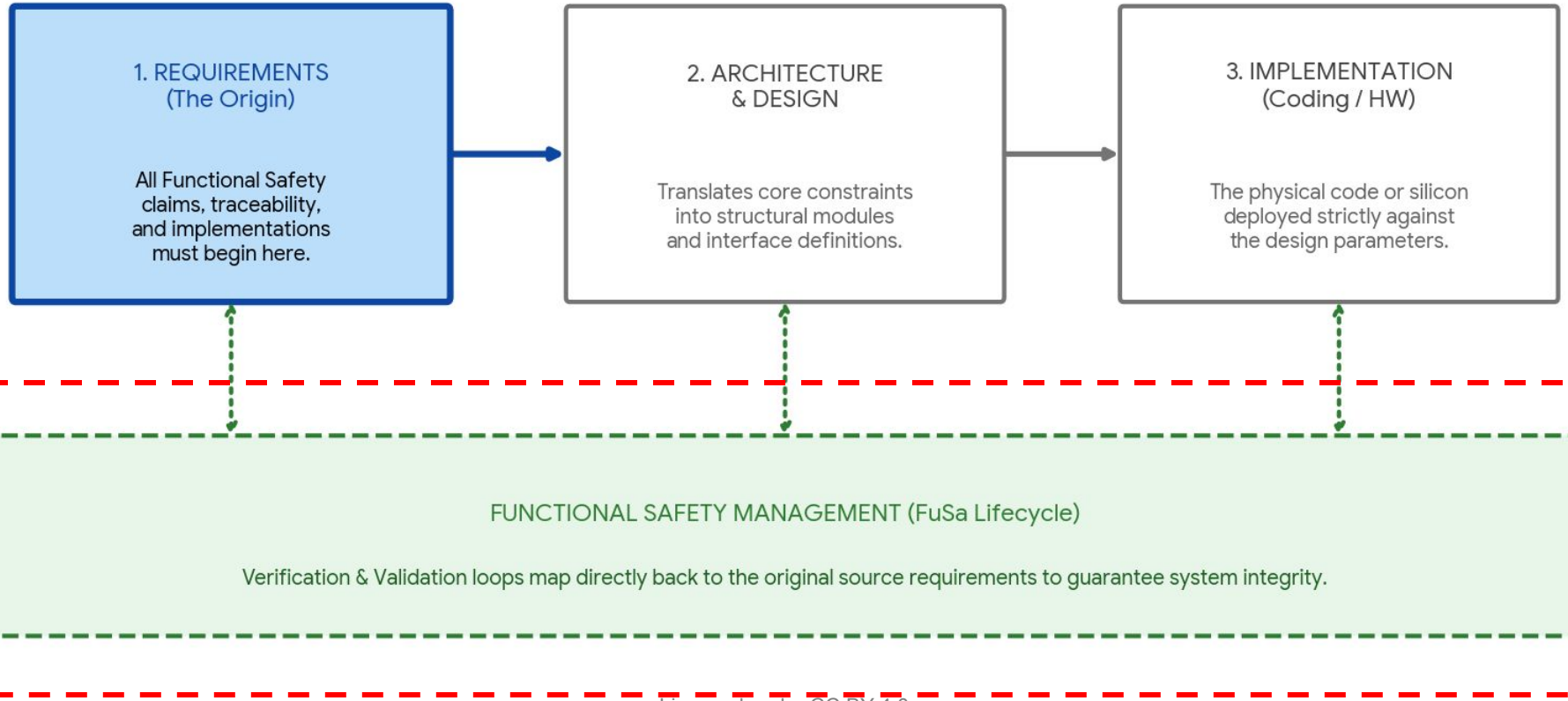
Architecture Spec

- While requirements describe the features that Xen provides, architecture spec describes how the feature is implemented, interfaces to the other architectural components, error propagation, data/control flow, etc
- Architecture Spec is linked to a set SSRs belonging to a component
- By linking SSRs to Arch spec , we **avoid** putting any markers in the code (and thereby ensure the code is clean).
- https://gitlab.com/xen-project/fusa/architecture_specs/-/blob/master/domain_creation_and_runtime/domain_partially_emulated_resources/arm64/generic_timer.rst

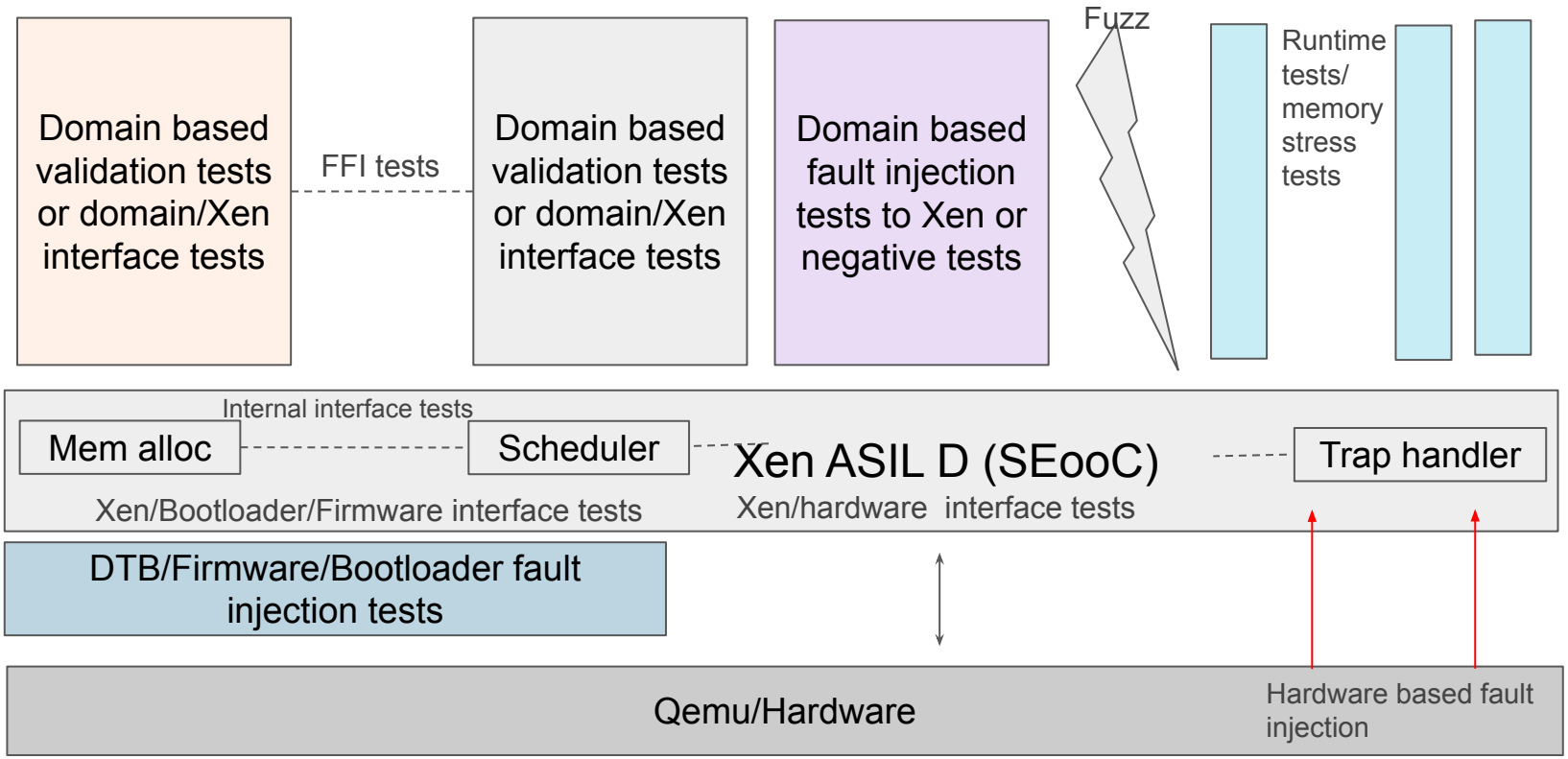




How do you verify and validate a complex system?



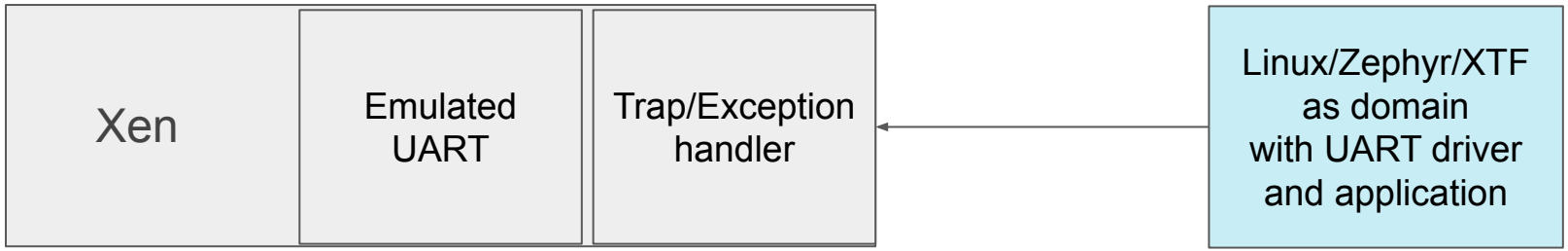
Overall Testing deployment



Domain based Tests/SSR tests/Xen domain interface

Domain based tests are used to verify the blackbox requirements or perform fault injection from domain

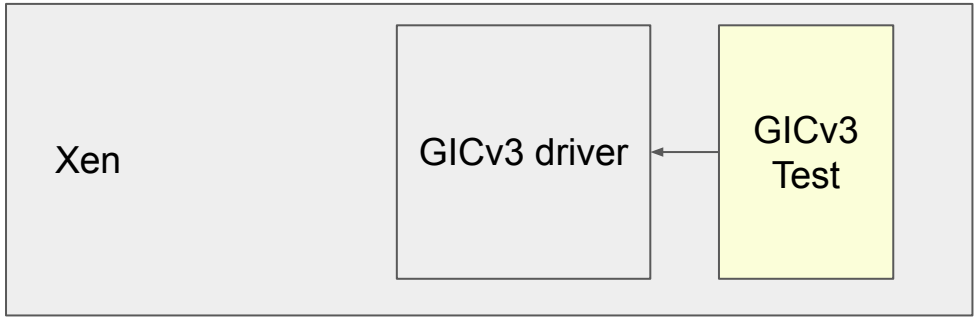
For eg - Xen shall support transmission of data in polling mode for the domain.



Pros - Xen is unmodified. It is tested in the way it will be used in production.
Cons - The tests are relatively high level. They can test a lot of code together.

Xen/HW interface Test

The test aims to validate Xen's configuration of the hardware. For eg Xen's GICv3 driver should be able to generate software generated interrupt
For this, we introduce self tests within Xen.

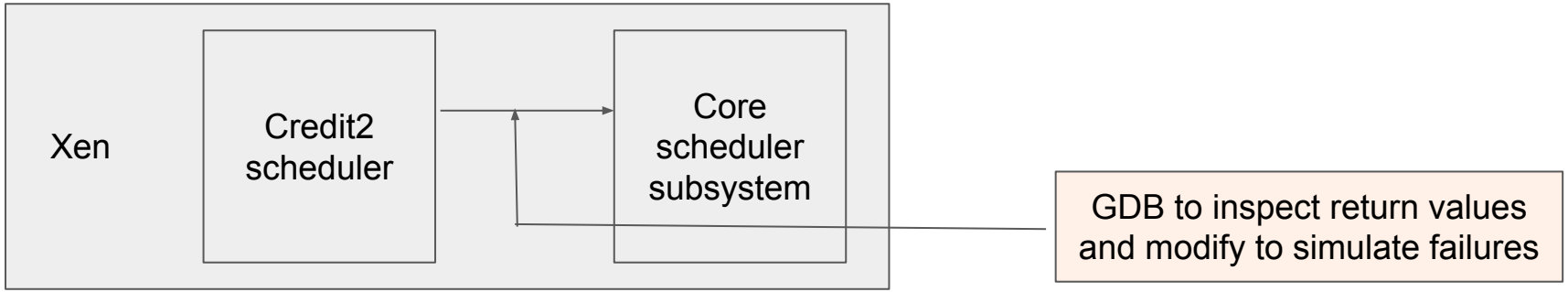


The test can be compiled out in the production build
The test runs during boot time

- https://gitlab.com/xen-project/fusa/test_specifications/-/blob/master/hw_sw_interface/test0_sgi_gicv3.rst
- https://lists.xenproject.org/archives/html/xen-devel/2025-09/msg00956.html

GDB based tests for internal interface

Xen has lots of internal components which invoke one other. We need a way to test the interface between the components. Also, we need a way to inspect the internal data structure and mock the error handling paths.

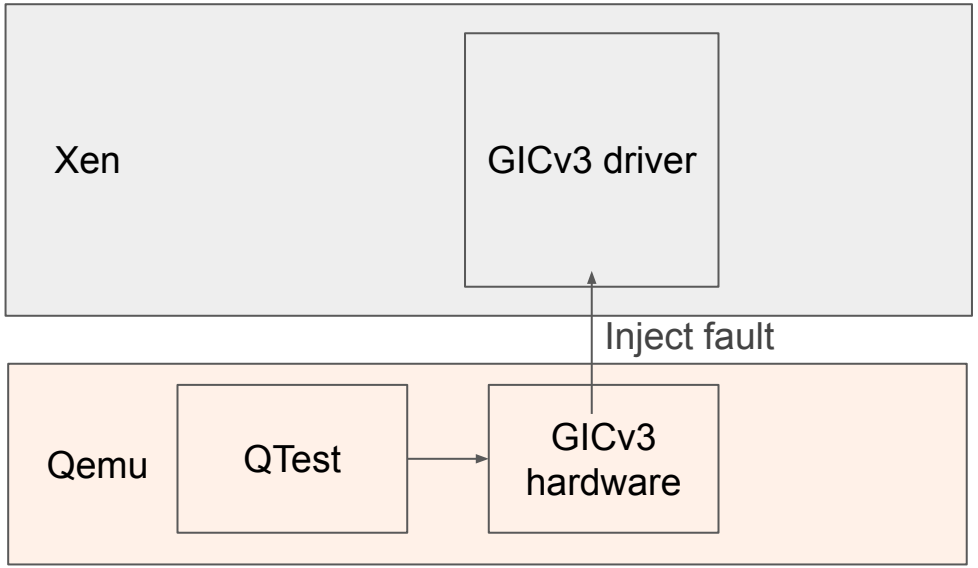


Use GDB to validate the internal data structures or the interface between different arch components

https://gitlab.com/xen-project/fusa/test_specifications/-/blob/master/domain_creation_and_runtime/scheduling/test_sched_core_halt_on_scheduler_failure.rst

Hardware Fault Injection using Qemu

Using Qemu testing framework

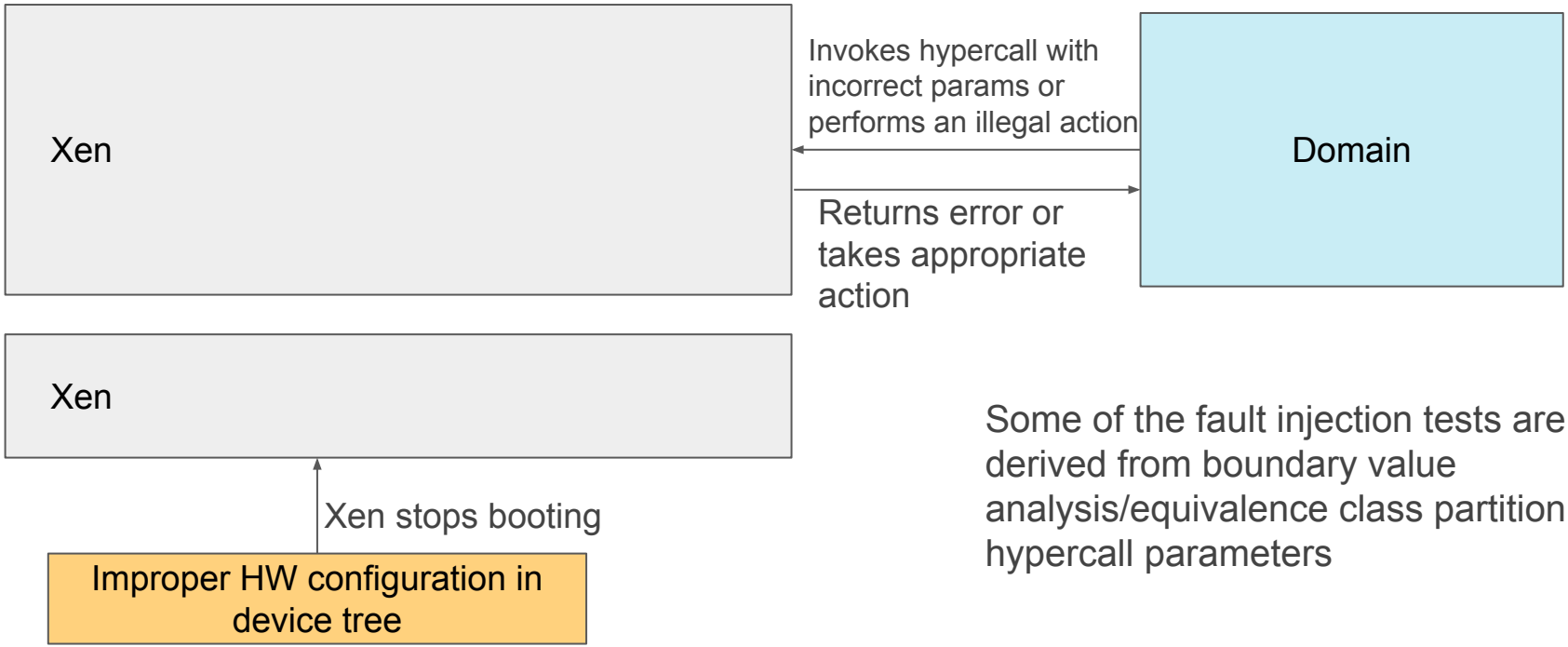


Xen panics

Qemu can emulate most of the hardware faults which are observed by Xen
It emulates many of AE IPs which have test registers

Software fault injection/negative tests

Using domains, device tree binaries, etc



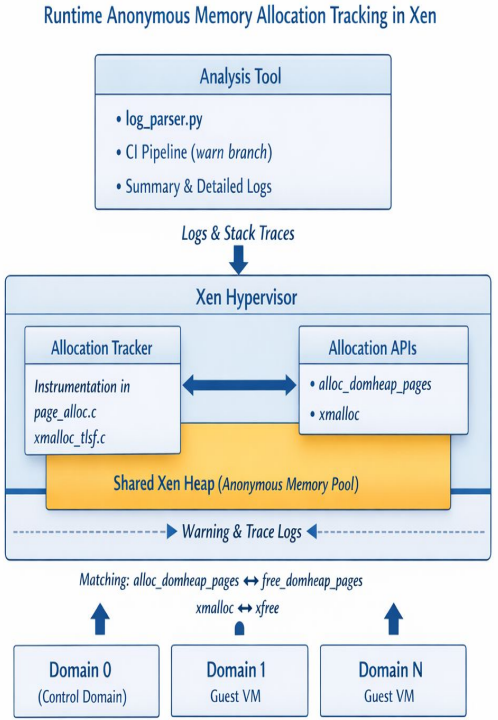
Some of the fault injection tests are derived from boundary value analysis/equivalence class partition of hypercall parameters

Runtime tests

Minimise the runtime memory allocation. To prevent Xen from running out of memory

- Xen has a **shared heap** → source of *anonymous allocations*
- Domains (d0, d1, …) trigger allocations indirectly via Xen
- Tool instruments **core allocators (TLSF + page allocator)**
- Runtime → **after `SYS_STATE_active` (post-boot steady state)**
- Output → **logs + stack traces + domain attribution + matching frees**

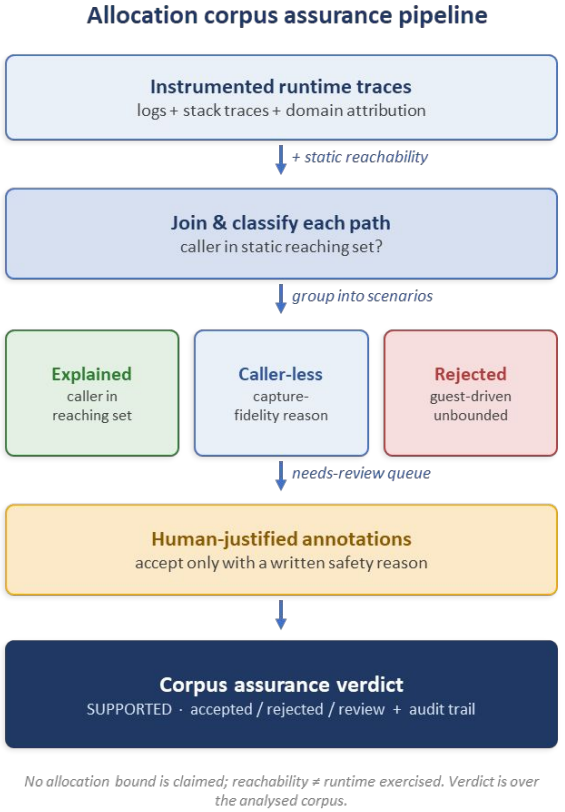
https://gitlab.com/minervasys/public/xen/-/blob/minerva/warn/minerva_analysis/README.md



From logs to a corpus assurance

Runtime traces turned into an auditable **safety claim**

- **Static reachability join** — every runtime allocation is matched against which callers can *statically reach* its allocator
- **Canonical scenarios** — duplicate stacks collapse into **one reviewable scenario** each
- **Cause / effect safety model** — accept only if the cause is *bounded* or the effect is *freed before return*
- **Curated annotations** — acceptance requires a **human justification** (never auto-accepted)
- **Honest residue** — caller-less & softirq/RCU paths labelled by capture-fidelity reason, not guessed
- **Verdict + audit trail** — accepted / rejected / needs-review with a reduction report



Going from Out of Context to “In context”

DO-178C certification is within the context of a system**. It consists of objectives which are met by completing activities (possibly with independence) to produce specific artifacts. This complements FUSA efforts as follows.

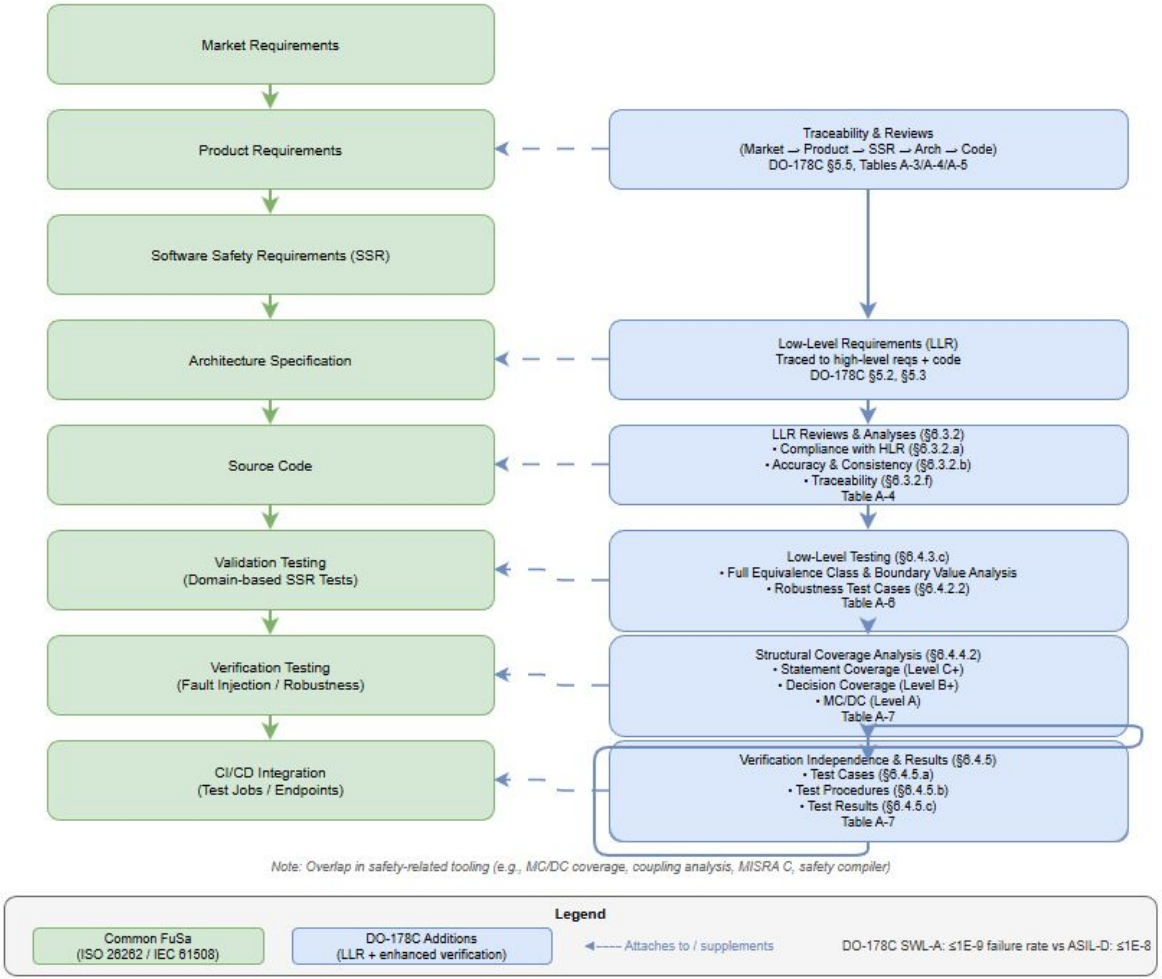
- **More stringent system failure target** - Catastrophic failure** conditions (driving SWL-A) require ≤1E-9 per flight hour vs ASIL-D ≤1E-8 per hour

- **Deeper verification rigor** - Strengthens the certification evidence

- **Evergreen-friendly** - Supports continuous upstream development; certification artifacts remain maintainable as the codebase evolves

**system-level concern from ARP 4754A/4761 drives this and sets the SWL

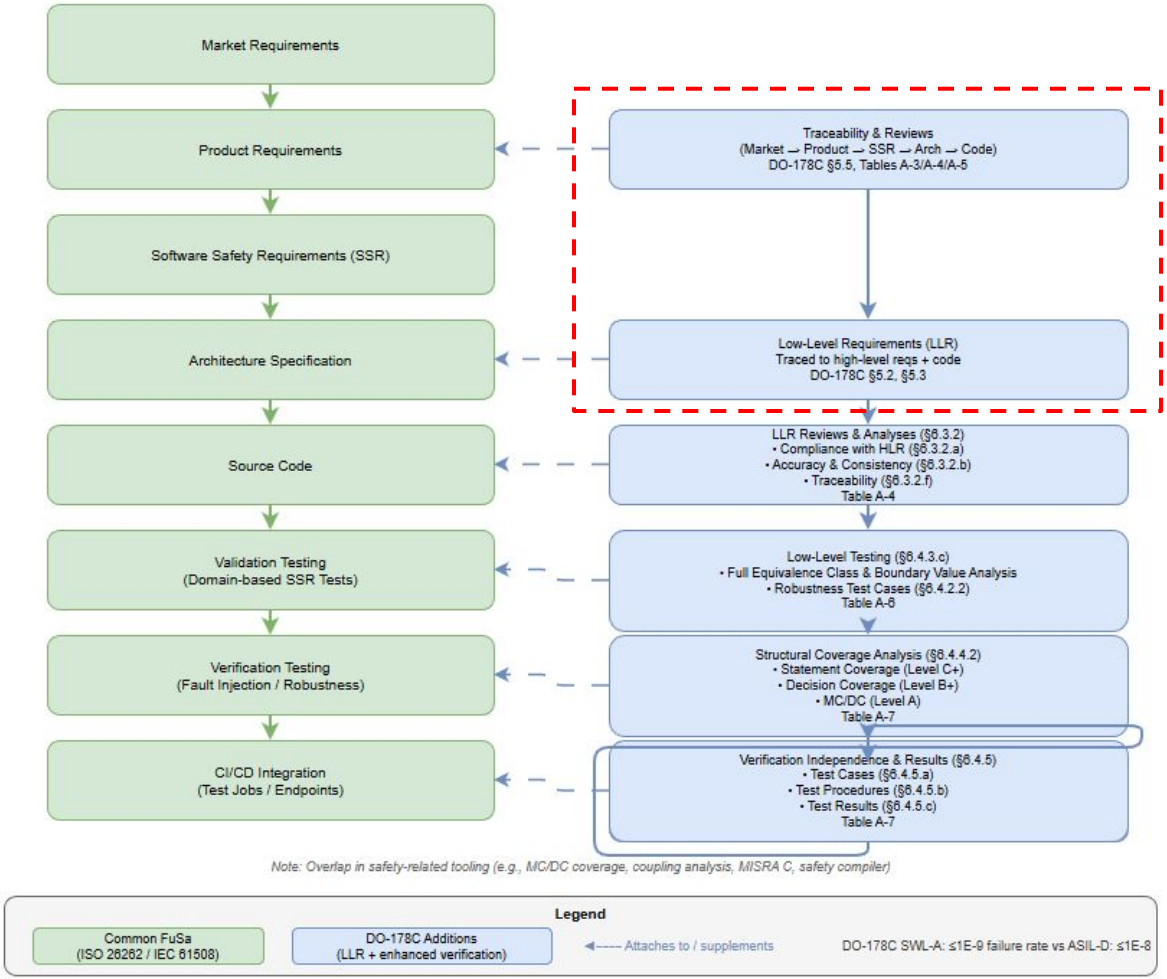
Additions to the FuSa Software Lifecycle



Additions to the FuSa Software Lifecycle:

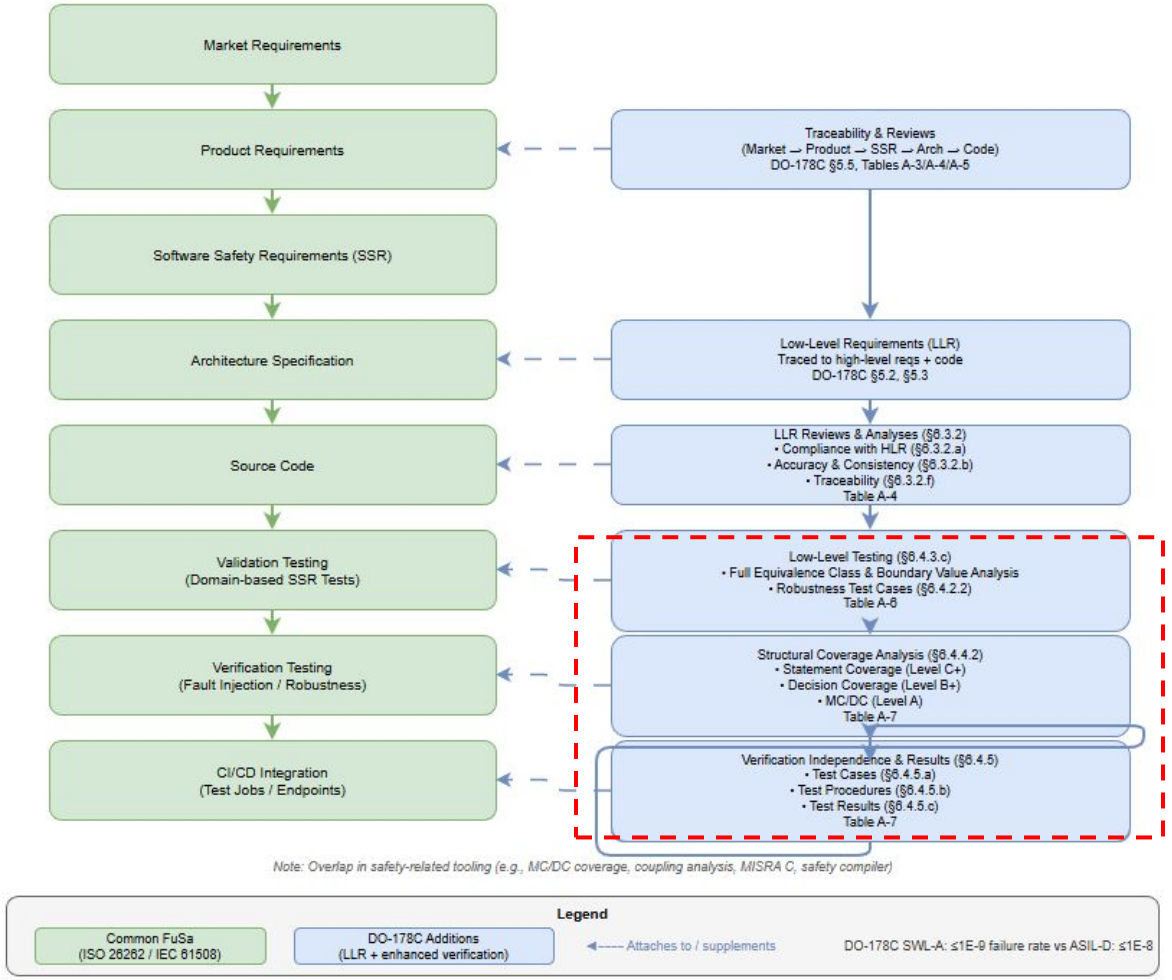
Low Level Requirements

https://gitlab.com/xen-project/fusa/requirements/-/tree/master/design_reqs/domain_creation_and_runtime/physical_resources/arm64/mmu/pt



Additions to the FuSa Software Lifecycle:

Unit Testing of LLRs for Coverage



https://gitlab.com/xen-project/fusa/test_specifications/-/blob/master/domain_creation_and_runtime/physical_resources/arm64/mmu/pt/pt.tc.md

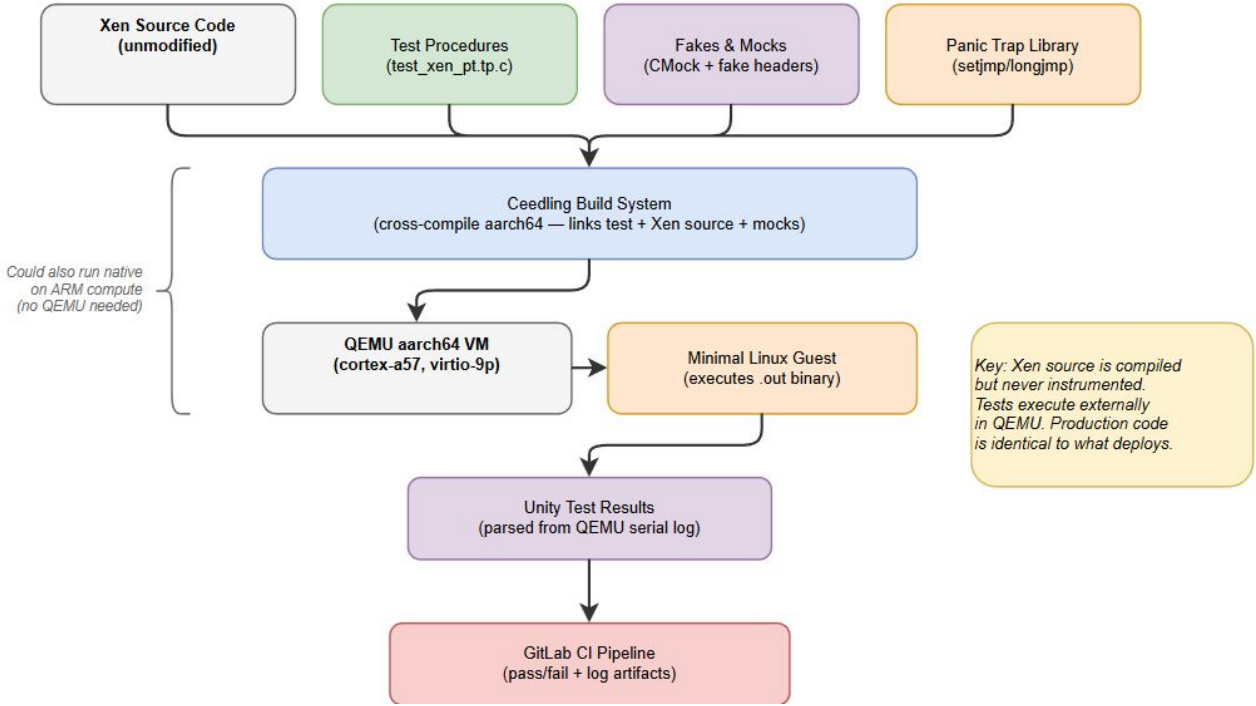
Unit testing(whitebox) of Xen Page Table

Goals:

Automated regression signals - Coverage and CI results provide early indicators of impactful changes

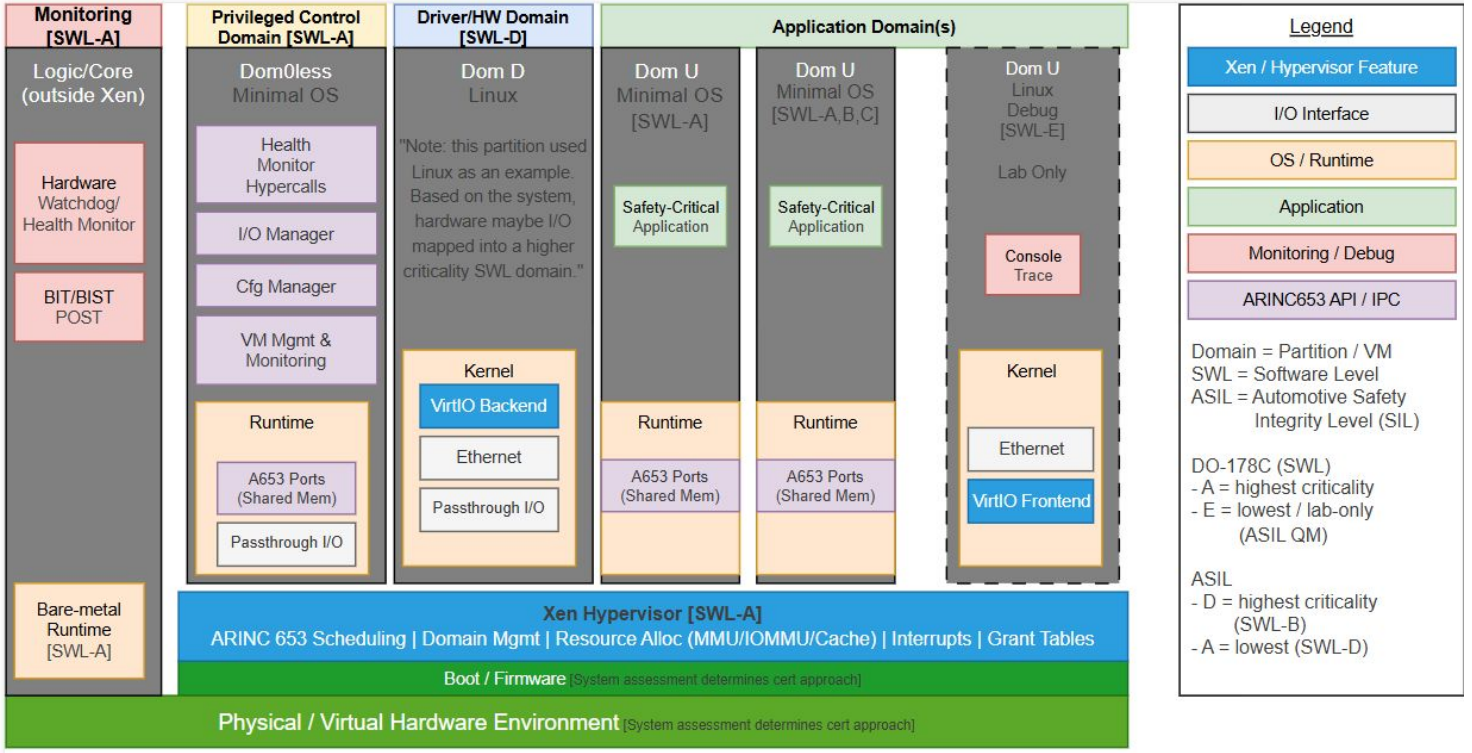
Non-intrusive testing - Validates and measures coverage without modifying production code, keeping the software under test identical to what deploys

Massive Scale - Supports thousands of low level tests, fails fast, and minimizes required compute time



https://gitlab.com/xen-project/fusa/xen-integration/-/merge_requests/20

Example: DO-178C Mixed Criticality System



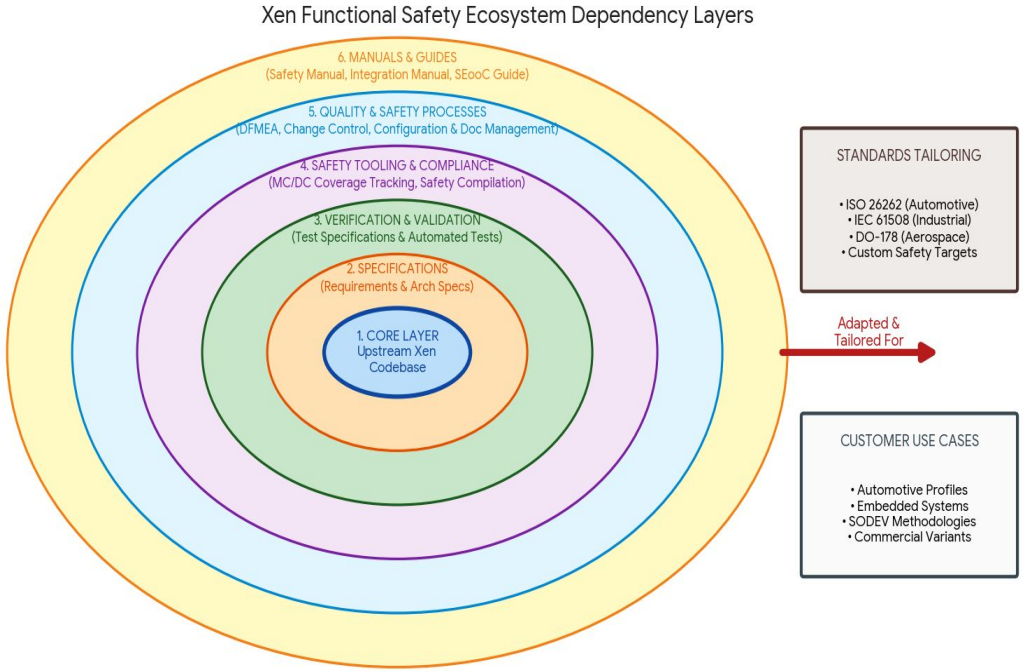
(DO-178C Sec. 2.3) *"only partitioned (time&space) software components can be assigned individual SWL by the system safety assessment Process."*

(DO-178C Sec. 2.4) *"if partitioning and independence between software components cannot be demonstrated, all components are assigned the SWL associated with the most severe failure condition to which the software can contribute"*

Diagram from: ELISA Aerospace WG discussions on mixed criticality - https://github.com/elisa-tech/wg-aerospace/pull/177

Adding and Maintaining evergreen certification artifacts

Launching a new Xen Project membership Tier for Safety
Defining the Quality Management processes & maintainability



The entire verification and compliance envelope surrounding the core Xen codebase can be custom-tailored to satisfy unique industry certification rules and target architectures.

- Lightweight QMS process
  - Approach: reStructuredText git-based docs-as-code
  - OpenFasttrace as external traceability engine
  - Sphinx for auditable living documentation rendering
- Mapping existing Xen community processes to standards
  - Use lightweight ISO/IEC 12207 for supporting processes
  - FuSa SIG implements limited & tailiored safety processes

Collaboration

To develop an open source, upstream safe hypervisor requires a lot of community effort

- Join xen-devel mailing list – All discussion. Reviews happen on the mailing list
- Join Xen Functional Safety matrix channel
- Calls every other Tuesday for members actively working on Xen FuSa
- Join us at Xen Summit – Sep 15-17, Munich, Germany

Thanks